Student Management System: File-by-File Learning Reference

This guide describes the files that make up this repository's working application. It follows the source as it exists here; it does not assume that features in the separate Module 6 reference ZIP are part of this project.

For the full system workflow, persistence design, glossary, and manual test steps, see PROJECT_STUDY_GUIDE.md.

1. How to use this guide

Start with the root files to learn how the app is installed and started. Then follow the runtime path:

```
client/index.html
  -> client/src/main.jsx
  -> client/src/App.jsx
  -> page component
  -> service module
  -> client/src/services/api.js
  -> server middleware and route
  -> server/utils/db.js
  -> JSON response to React
```

The comments in source files explain important lines or related groups of lines. They are not meant to repeat every JSX tag, CSS property, or punctuation mark: doing so would obscure the logic. JSON manifests and lockfiles cannot contain JavaScript-style comments without becoming invalid JSON.

1a. What the Module 6 reference actually demonstrates

The archive contains 25 project-owned `src` files, one project test file, and one `public/.htaccess` file when its bundled `node_modules` are excluded. Its commenting is not uniform:

- **React/JavaScript source:** Many components and layouts begin with a short

file-purpose comment, with extra comments around selected state, routing, authentication and rendering logic. Other files, including the route test, have no explanatory comments. The ZIP therefore demonstrates useful section-level teaching comments, not a rule that literally every line is commented.

- **CSS:** Comments introduce style groups and some individual layout rules.

They help a reader connect selectors to page areas, while the CSS syntax itself remains uncommented line by line.

- **Test:** `tests/test-routes.mjs` reads application files and uses Node's

strict assertions to check expected route/auth/progress text. It contains no comments, so its behavior must be understood from its variable names and assertions.

- **Public rewrite configuration:** `public/.htaccess` uses short comments to

explain Apache behavior: leave existing files/directories alone and route other paths to the single-page app's HTML. It is web-server configuration, not React code.

The Student Management System uses the same helpful patterns where suitable: file-purpose comments, logical-section notes, and explanations at non-obvious decisions. Its architecture differs, so its comments describe its own React client, Express API, account ownership and JSON persistence rather than copying unrelated course-enrollment explanations.

2. Repository-level files

`README.md`

- **Purpose:** The short entry point for a developer opening the repository.
- **Responsibilities:** Lists the technology stack, local setup, environment configuration, test command, persistence behavior, and production caveats.
- **Connections:** Links to the project study guide; its commands invoke scripts defined in the root `package.json`.
- **Workflow and impact:** Read this first to install and start the app. It correctly notes that the small JSON store is intended for the capstone/demo, not a replacement for a production database.

`package.json`

- **Purpose:** Defines commands and packages for the complete repository.
- **Responsibilities:** `dev` starts the API and Vite client together using `concurrently`; `build` builds the client; `test` runs client and server suites; `postinstall` installs dependencies in both subprojects.
- **Dependencies:** Root packages include tools used across the workspace. The client and server also have their own manifests.
- **Example:** `npm run dev` runs the two processes in parallel so the browser and API are both available during development.

`package-lock.json`

- **Purpose:** Records the exact dependency versions selected by npm.
- **Responsibilities:** Makes installs more repeatable across computers.
- **Connections:** Generated from `package.json`; do not hand-edit it or add comments to it.

`.gitignore`

- **Purpose:** Tells Git which local or generated files should not be committed.
- **Responsibilities:** Excludes dependency folders, build output, `.env` configuration, logs, and `server/data/db.json`.
- **Security and impact:** This helps prevent local secrets and personal records from being committed accidentally. It is still important to inspect `git status` before sharing code.

`docs/PROJECT_STUDY_GUIDE.md`

- **Purpose:** Gives a beginner-oriented explanation of the architecture, persistence and complete application workflows.
- **Responsibilities:** Covers data isolation, refresh/login behavior, testing, troubleshooting, extension ideas, and terminology.
- **Connections:** Use it with this per-file guide: the study guide explains

cross-file behavior; this guide explains each file's individual role.

`docs/PROJECT_FILE_REFERENCE.md`

- **Purpose:** This document: a directory-by-directory guide to the actual source, tests, settings and safe environment configuration.
- **Responsibilities:** Provides a logical code walkthrough and links related files so a learner can follow the same operation across the application.

Other root documentation

- **`INDEX.md`:** Navigation page for the documentation set.
- **`IMPLEMENTATION_SUMMARY.md`:** Overview of the persistence work and its verification.
- **`PERSISTENCE_IMPLEMENTATION.md`:** More technical notes on ownership, progress records and storage behavior.
- **`TESTING_GUIDE.md`:** Manual test steps for signing up and verifying that data returns after refresh and login.
- **`START_HERE.md`:** Quick-start summary. It should be treated as a convenience document; use the source and current test output as the final authority on behavior.
- **`module-6-tech-academy.zip`:** Reference material supplied for comparison; it is not a runtime dependency of the student-management app.

3. Client application files

The browser app lives under `client/`. Components describe the interface, services perform HTTP requests, and `AuthContext` shares the current account and saved workspace state.

HTML, startup, routes and styles

`client/index.html`

- **Purpose:** The HTML shell Vite serves to the browser.
- **Walkthrough:** The document declares language and metadata, creates the empty `#root` element, then loads `/src/main.jsx` as a JavaScript module.
- **Connections:** React takes over the root element in `main.jsx`.
- **Example:** Without `#root`, `createRoot(...)` would have no place to mount the interface.

`client/src/main.jsx`

- **Purpose:** Starts the React interface.
- **Walkthrough:** Imports React, the DOM renderer, router, app routes, progress tracker, authentication provider and global CSS. `createRoot(...).render(...)` attaches the component tree to the HTML root. `StrictMode` helps expose unsafe development side effects; `BrowserRouter` enables URL navigation; `AuthProvider` shares account state; `ProgressTracker` saves route changes; `App` renders the matched page.
- **Connections:** This is the bridge from `index.html` to `App.jsx`.

`client/src/App.jsx`

- **Purpose:** Declares the browser URL map.
- **Walkthrough:** Imports router components and common layouts. Each `lazy`

call loads a page only when needed. `Suspense` displays a loader until that page's JavaScript arrives. Public `/login` and `/register` routes use `AuthLayout`; student and course routes are nested under `ProtectedRoute` and `DashboardLayout`; `/` redirects to the dashboard; `*` catches unknown paths.

- **Connections:** The page modules under `client/src/pages/` supply the route

content; layout components provide shared frames.

`client/src/index.css`

- **Purpose:** Defines the visual system and responsive page layout.
- **Walkthrough:** Global tokens and browser defaults come first, followed by

shell/navigation, headings/buttons, dashboard cards, tables and status indicators, forms, student/course details, modal, and login/register styles. Media queries at 1100px, 800px and 520px progressively reduce columns, turn the sidebar into a mobile drawer, stack forms/actions, and reduce small-phone spacing.

- **Connections:** JSX uses the class names defined here.
- **Impact:** CSS affects presentation and breakpoints, not record persistence

or authorization.

Shared context and reusable components

`client/src/context/AuthContext.jsx`

- **Purpose:** Central account/session and resumable-progress state.
- **Imports:** React context/hooks, `authService`, `progressService`, and the

shared error formatter.

- **Walkthrough:** Creates the context; `AuthProvider` initializes the cached

profile and loading/progress flags; a ref stores a serialized save queue and another tracks the active account. `refresh` validates an existing token against `/auth/me` and fetches that user's progress. The initial effect calls it. `login` and `register` establish a server session, fetch progress, then set the user. `logout` clears browser session and in-memory private state. `saveProgress` immediately updates React state, queues an authenticated save, checks the account has not changed, and records/display errors. `useMemo` shares a stable provider value. `useAuth` gives components access to the context and throws a helpful error when used outside the provider.

- **Connections:** Used by protected routes, layouts, auth pages, form pages,

progress tracker and student search.

- **Example:** A student form calls `saveProgress`; the provider updates the

screen immediately and sends a queued update to the server.

`client/src/components/ProgressTracker.jsx`

- **Purpose:** Invisible watcher that remembers the current private route.
- **Walkthrough:** Reads the router location and auth context; an effect waits

until session and progress loading finish, skips public auth paths and already-saved paths, then saves a changed pathname.

- **Connections:** Mounted once in `main.jsx`; uses `saveProgress` from

`AuthContext`.

`client/src/components/ProtectedRoute.jsx`

- **Purpose:** Stops signed-out users from viewing private screens.
- **Walkthrough:** Reads authentication/loading state and current location.

While checking a saved session it renders a loader. If not signed in, it redirects to `/login` and stores the original path in router state. Otherwise `<Outlet />` renders the protected child route.

- **Security note:** This is a user-interface gate only. Server middleware

independently validates every private API request.

`client/src/components/ConfirmModal.jsx`

- **Purpose:** Reusable confirmation dialog before a destructive action.
- **Walkthrough:** Receives title/message/button text and callbacks as props.

The backdrop cancels; mouse events inside the dialog are stopped from bubbling to the backdrop. Dialog ARIA attributes name the popup. `busy` disables both buttons during a request.

- **Connections:** Used by course and student list/detail pages.

`client/src/components/EmptyState.jsx`

- **Purpose:** Explains when a successful list request has no records.
- **Walkthrough:** Displays the supplied title and message, and optionally the

action element supplied by the parent page.

- **Distinction:** It is not an error state; a blank list can be a valid result.

`client/src/components/ErrorState.jsx`

- **Purpose:** Presents a failed-load message and optional retry button.
- **Walkthrough:** `role="alert"` makes new error text available to assistive

technology; `onRetry` is called only when the parent provides it.

- **Connections:** Dashboard, course and student pages provide their load

callbacks.

`client/src/components/FormField.jsx`

- **Purpose:** Keeps form labels, required markers and validation messages

consistent.

- **Walkthrough:** Receives the label, field name, error, required flag and

child input. The `<label htmlFor>` points to the matching input id; `children` lets the parent choose an input, select or textarea.

- **Connections:** Login, registration and create/edit forms.

`client/src/components/Loader.jsx`

- **Purpose:** Shared accessible loading indicator.
- **Walkthrough:** A `fullScreen` prop selects CSS presentation, `label` selects

its text, and `role="status"` announces loading.

- **Connections:** Used for page data loads, lazy page downloads and session

validation.

`client/src/components/StatCard.jsx`

- **Purpose:** Displays one dashboard statistic.
- **Walkthrough:** Receives label, value, detail and icon; renders them without

fetching data itself.

- **Connection:** `Dashboard.jsx` calculates values and supplies the props.

Layouts and pages

`client/src/layouts/DashboardLayout.jsx`

- **Purpose:** Shared frame around signed-in screens.
- **Walkthrough:** Defines navigation items, reads current user/logout/progress

error from context, and stores mobile drawer visibility in local state. `handleLogout` clears the session then replaces navigation with login. Links are generated from an array; mobile overlay closes the drawer. The header shows account details, progress errors appear consistently, and `<Outlet />` displays the current nested page.

- **Connections:** Parent of dashboard, student and course routes in `App.jsx`.

`client/src/pages/AuthLayout.jsx`

- **Purpose:** Shared public shell for sign-in and registration.
- **Walkthrough:** Waits for session restoration; an already-authenticated user

is redirected into the dashboard. `<Outlet />` inserts login or registration content.

`client/src/pages/Login.jsx`

- **Purpose:** Collects credentials and starts an existing account session.
- **Walkthrough:** Controlled `values` state mirrors typed email/password;

errors, server error and busy state are separate. `submit` prevents native page reload, checks email/password, calls context `login`, then navigates to the original guarded route, saved route, or dashboard. `FormField` pairs labels/errors with inputs; password type masks the entry.

- **Connections:** `AuthContext` -> `authService` -> `/api/auth/login`.

`client/src/pages/Register.jsx`

- **Purpose:** Creates a new user account.
- **Walkthrough:** Controlled name/email/password/confirmation values are

validated by `validateRegistration`. Each password requirement is evaluated for a live checklist. On valid submission, `register` creates and signs in the user, then the page navigates to the dashboard; failures are displayed.

- **Connections:** `validation.js`, `FormField`, `AuthContext` and the auth API.

`client/src/pages/NotFound.jsx`

- **Purpose:** Friendly screen for a URL that has no route.
- **Walkthrough:** Provides a link back to the dashboard; selected by the

wildcard route in `App.jsx`.

`client/src/pages/Dashboard.jsx`

- **Purpose:** Overview for the authenticated academic workspace.
- **Walkthrough:** Loads students and courses together with `Promise.all`;

tracks loading and error state; derives active student count and average score from the returned records; renders stat cards, five recent students, and links for common create actions.

- **Connections:** `studentService`, `courseService`, `StatCard`, `ErrorState`,

`Loader`, and the current profile from `AuthContext`.

`client/src/pages/students/Students.jsx`

- **Purpose:** Search and manage the current account's student records.

- **Walkthrough:** Exports `getScoreIndicatorClass` for score-range styling and

tests. The component initializes the search query from restored progress, loads the list, debounces progress saves by 400ms, derives a filtered list, and manages delete confirmation and status. It displays loading/error, empty/no-match, or table states; row links open detail/edit routes.

- **Connections:** `studentService`, shared state/components and progress API.

`client/src/pages/students/StudentForm.jsx`

- **Purpose:** Creates or edits a student, and restores unfinished values.

- **Walkthrough:** Route `id` chooses edit mode. The form loads available

courses and (when editing) the existing student. It overlays only a draft belonging to the same record id. State stores field values/errors/loading. A 400ms timer saves the draft. Submit validates, converts score to a number, calls create/update, clears the completed draft, and returns to the list.

- **Connections:** `studentService`, `courseService`, validation, auth progress

and reusable form/loader components.

`client/src/pages/students/StudentDetails.jsx`

- **Purpose:** Shows one student's profile and academic information.

- **Walkthrough:** Reads `id` from the route and loads that student. Edit links

open the form; delete first opens `ConfirmModal`, then calls the service and navigates to the list. Errors and loading use shared components.

- **Connections:** Student service, route parameters and reusable UI.

`client/src/pages/courses/Courses.jsx`

- **Purpose:** Lists and deletes courses belonging to the current user.

- **Walkthrough:** Loads records, tracks error/loading, shows an empty state or

course cards, and stores a selected `deleteId`. A confirmation calls `courseService.remove`; server conflict errors are shown if students still reference a course.

- **Connections:** Course service, shared state components and routes.

`client/src/pages/courses/CourseForm.jsx`

- **Purpose:** Creates/edits a course and autosaves unfinished values.

- **Walkthrough:** Route `id` selects create or edit. Edit loads the server

course, then overlays a matching saved draft. The effect debounces draft saves; submit validates, calls create/update, clears its draft and navigates to the course list.

- **Connections:** Course service, `validateCourse`, `AuthContext`, form and

loader components.

`client/src/pages/courses/CourseDetails.jsx`

- **Purpose:** Displays a course's description, creation date and student count.

- **Walkthrough:** Reads `id`, loads the course, provides retry on failure, and

links to edit/course list/student list.

- **Connection:** `courseService.getById` returns the server's decorated course.

Client services and utilities

`client/src/services/api.js`

- **Purpose:** Shared Axios configuration for all HTTP calls.

- **Walkthrough:** Creates an Axios instance using `VITE_API_URL` or the local

API default. A request interceptor attaches the saved bearer token. A response interceptor clears stale local credentials after HTTP 401 and rejects the promise so pages can display an error.

- **Security:** Vite `VITE_` variables are compiled for browser use. They must

never contain private keys or server secrets.

`client/src/services/authService.js`

- **Purpose:** Wraps login/register/profile HTTP requests and browser session

storage.

- **Walkthrough:** `store` writes token and safe profile to localStorage;

`login`/`register` call auth endpoints and store successful responses; `getMe` fetches the server-verified profile; `logout` removes local keys; accessors read the token/profile; parsing errors in cached profile return `null`.

- **Connections:** Used by `AuthContext` and `api.js`.

`client/src/services/studentService.js`

- **Purpose:** Small wrapper around student endpoints.

- **Walkthrough:** Provides list (optional query parameters), detail, create,

update and remove methods. Each unwraps Axios response data so pages work with student records rather than transport response objects.

`client/src/services/courseService.js`

- **Purpose:** Wrapper around course list/detail/create/update/delete routes.

- **Walkthrough:** Same pattern as `studentService`; centralizes URL and

response-envelope details for pages.

`client/src/services/progressService.js`

- **Purpose:** Gets and updates account-owned saved workspace state.

- **Walkthrough:** `get` calls `GET /progress`; `update` sends a partial object

to `PUT /progress`. The server derives account identity from the token.

`client/src/utils/errors.js`

- **Purpose:** Converts Axios/server/JavaScript errors into displayable text.

- **Walkthrough:** Prefers a server response message, then a JavaScript

`message`, then a caller-supplied fallback.

`client/src/utils/validation.js`

- **Purpose:** Shared browser-side form checks.

- **Walkthrough:** Exports email pattern, named password requirement functions,

aggregate password check, and validators for student/course/registration. Validators return objects keyed by input name, allowing the form to show an error beside its field.

- **Security:** These checks improve feedback but do not replace server validation.

Client tests and tool configuration

`client/src/tests/setup.js`

- **Purpose:** Loads jest-dom matchers before each client test suite.
- **Impact:** Enables readable assertions such as `toBeInTheDocument()`.

`client/src/tests/api.test.js`

- **Purpose:** Unit test for the student service list method.
- **Walkthrough:** Mocks Axios, arranges a fake list response, asserts the service unwraps the student array and calls the expected endpoint.

`client/src/tests/ProtectedRoute.test.jsx`

- **Purpose:** Verifies the route guard's waiting state.
- **Walkthrough:** Mocks auth context and uses `MemoryRouter` rather than a real browser; confirms that a status/loading element appears during session validation.

`client/src/tests/validation.test.js`

- **Purpose:** Tests score styling boundaries, invalid student values, and registration password rules.
- **Impact:** Prevents changes to helper logic from silently weakening these expected behaviors.

`client/vite.config.js`

- **Purpose:** Configures Vite and Vitest.
- **Walkthrough:** Enables React/JSX transformation, fixes the dev port, uses a jsdom test environment, and loads `src/tests/setup.js`.

`client/eslint.config.js`

- **Purpose:** Configures static code checks.
- **Walkthrough:** Ignores generated `dist`, enables recommended JavaScript rules, declares browser/Node globals, parses JSX/modules, and applies React hooks and refresh rules.

`client/package.json` **and** `client/package-lock.json`

- **Purpose:** Define client-specific scripts, packages and locked versions.
- **Scripts:** `dev` starts Vite, `build` produces `dist`, and `test` runs Vitest. The lockfile records exact dependency resolution and should be regenerated by npm rather than hand-edited.

4. Server application files

The server is a Node.js/Express API. It is the trusted side of the application: it validates tokens, decides ownership, validates data and writes durable records.

Startup, Express setup and authentication

`server/server.js`

- **Purpose:** Opens the HTTP listener.
- **Walkthrough:** Imports configured Express `app`, chooses `PORT` or 5000,

then calls `listen`. It contains no route logic.

`server/app.js`

- **Purpose:** Assembles the API in a testable Express application.
- **Walkthrough:** Loads dotenv; requires a configured JWT secret in production

and supplies a development-only fallback outside production; creates Express; enables CORS and JSON parsing; registers a public health route and auth router; mounts student/course/progress routes behind `requireAuth`; finishes with 404 and error responses.

- **Connections:** Imported by `server.js` and by integration tests.

`server/middleware/auth.js`

- **Purpose:** Verifies identity for private routes.
- **Walkthrough:** Reads the `Authorization: Bearer ...` header. Missing tokens

receive 401. `jwt.verify` checks signature and expiration; the matching user must still exist in the JSON database. The middleware puts only id/name/email on `req.user`, then calls `next()` so the route can proceed. Invalid/expired tokens receive 401.

- **Security:** Downstream ownership checks use this verified identity, never a

client-supplied owner id.

`server/routes/auth.js`

- **Purpose:** Defines register, login and current-profile endpoints.
- **Walkthrough:** Registration checks required fields and password rules,

calls `register`, and returns 201. Login requires email/password and calls `login`. Service errors map to appropriate HTTP statuses. `/me` uses auth middleware and returns its trusted profile.

- **Connection:** Account persistence and token work live in

`services/authService.js`.

`server/services/authService.js`

- **Purpose:** Implements account creation and credential verification.
- **Walkthrough:** `publicUser` omits the hash. `tokenFor` signs the user's id

into JWT `sub` with a two-hour expiry. `register` normalizes email, hashes password with bcrypt, then checks duplicate email and inserts the user inside serialized `updateDb`. `login` finds normalized email and compares password with bcrypt; failure returns one generic invalid-credentials error.

- **Impact:** Passwords are never stored as plain text or returned to clients.

Academic and progress routes

`server/routes/courses.js`

- **Purpose:** REST routes for courses.
- **Walkthrough:** `decorate` calculates student count for the same owner;

`validate` checks required values. GET list/detail filter by owner. POST normalizes code, checks per-user duplicates, assigns authenticated `ownerId`, and inserts transactionally. PUT checks both course id and owner, validates and checks duplicate code. DELETE returns not-found for a non-owned course and conflict if the same user's students still refer to it.

- **Connections:** Mounted at `/api/courses` after `requireAuth`.

`server/routes/students.js`

- **Purpose:** REST routes for student records.

- **Walkthrough:** `decorate` includes a course only when course and student

belong to the same account. `validate` checks name, email, course ownership, status and 0–100 score. GET list filters by owner before search; detail checks owner. POST/PUT validate and use `updateDb`; new rows receive server-generated UUID, verified `ownerId`, normalized email and timestamp. DELETE removes only a row owned by the requester.

- **Connections:** Mounted at `/api/students`; student forms use its service.

`server/routes/progress.js`

- **Purpose:** Saves and returns private resumable UI state.

- **Walkthrough:** Allowlisted route patterns prevent arbitrary resume URLs.

`cleanDraft` accepts null or a draft object and copies only recognized fields. `cleanChanges` validates route/search/draft fields into a fresh object. GET reads progress by `req.user.id`; PUT rejects invalid bodies, merges allowed partial changes into that user's progress, removes cleared drafts, and stores within `updateDb`.

- **Security:** The request body cannot choose the account whose progress is

being changed.

Database utility

`server/utils/db.js`

- **Purpose:** Reads/writes the persistent JSON database.

- **Walkthrough:** Builds a default file path relative to this module, while

allowing tests to provide `SMS_DB_PATH`. Defines empty collections and write queues. `readDb` parses JSON, adds the progress object when migrating older data, and creates a seed file only for `ENOENT`. `writeDb` serializes file writes, ensures the directory exists, writes a temporary complete document and renames it. `updateDb` queues the entire read-change-write sequence.

- **Limit:** Queues coordinate operations within one Node process; production

multi-process hosting should use a transactional database.

Server tests and package configuration

`server/tests/api.test.js`

- **Purpose:** Integration tests exercise the API through HTTP-like requests.

- **Walkthrough:** Sets `SMS_DB_PATH` to an isolated temporary file before

dynamically importing the app. Helpers register accounts, form bearer headers and create courses. Setup clears only that file; cleanup removes it. Tests cover health, registration/login and durable data, user isolation, progress validation, and course/student deletion constraints.

- **Impact:** Tests the real middleware/routes/database path without touching

the developer's normal `server/data/db.json`.

`server/package.json` **and** `server/package-lock.json`

- **Purpose:** Define server scripts and exact package versions.

- **Packages:** Express/CORS/dotenv provide the HTTP application and settings;

bcrypt/JWT implement password hashing and signed sessions; Vitest/Supertest support tests.

- **Scripts:** `dev` starts Node watch mode; `start` runs the server; `test`

executes Vitest.

`server/.env.example`

- **Purpose:** Safe template naming server configuration fields without a real

deployment secret.

- **Variables:** `PORT` selects the listener port (default 5000 if absent);

`JWT_SECRET` is a server-only signing key required in production. Replace the example placeholder outside source control before deployment.

- **Connections:** `server.js` reads `PORT`; `authService.js` signs tokens with

`JWT_SECRET`; `app.js` checks the production secret.

5. Environment configuration, safely

This repository contains a local `client/.env` and a tracked `server/.env.example`. This guide deliberately records variable names only and does not copy any local values. Do not share `.env` file contents publicly.

Variable	Location	Used by	Meaning and exposure
`VITE_API_URL`	`client/.env` (local)	`client/src/services/api.js`	API base URL. Vite exposes variables with the `VITE_` prefix in browser code, so it is configuration, not a place for credentials or secrets.
`PORT`	`server/.env` when configured; shown in `.env.example`	`server/server.js`	HTTP listener port. The code defaults to 5000 when unset.
`JWT_SECRET`	`server/.env` when configured; template shown in `.env.example`	`server/app.js`, `server/services/authService.js`, `server/middleware/auth.js`	Private server signing secret used to issue and verify login tokens. It must not have a `VITE_` prefix, be bundled into the client, committed, or copied into documentation.
`SMS_DB_PATH`	Optional test/runtime override	`server/utils/db.js`; set by server integration tests	Selects a database file path. Tests use a temporary path so they cannot erase real local data.
`NODE_ENV`	Deployment/runtime environment	`server/app.js`	When equal to `production`, startup refuses to use the development JWT-secret fallback.

The actual server `.env` is not present in this checkout; the example template is. The actual `client/.env` exists locally. The local values were not displayed, copied, or put in the downloadable archive. `.gitignore` excludes `.env` files and permits only the safe `.env.example` template.

6. Files that are not application source

- **`node_modules/`:** downloaded dependencies; generated by npm and not

beginner-authored project logic.

- **`client/dist/`:** built output generated by Vite; recreate with `npm run build`.
- **`server/data/db.json`:** runtime user/course/student/progress data; must not be distributed as sample source because it may contain private user data.
- **`client/.env`:** local configuration, excluded from the source archive.
- **Lockfiles:** important to reproducible installs, but generated JSON rather than hand-commented code.

7. End-to-end example: saving a student

1. `StudentForm.jsx` keeps typed input in React state and calls `validateStudent`.
1. `studentService.create` sends the object through the shared Axios instance.
2. `api.js` attaches the browser's token as a bearer credential.
3. `requireAuth` verifies that token, finds its account, and sets `req.user`.
4. `routes/students.js` validates all fields and checks the selected course is owned by the same account. It assigns `ownerId` from `req.user`, not from the form.
1. `updateDb` serializes the read/modify/write. `writeDb` writes and atomically replaces the JSON file.
1. The server returns the saved student. The client navigates to the list, and its next GET is filtered to that account.

The course and progress workflows follow the same pattern. Progress updates contain only allowed page/search/draft data and are keyed by the verified user id.

8. Existing features and boundaries

Implemented: account registration/login, JWT validation, private student and course CRUD, server-side ownership checks, student/course relationship validation, student search, resumable route/search/form drafts, responsive layouts and tests.

Not implemented: a separate task checklist or course-learning completion tracker, offline synchronization, multi-device refresh-token management, server-side JWT revocation, production database migrations, or multi-server transaction coordination. Do not infer these features from the Module 6 reference ZIP.